

Request Processor - Plain-Language Overview

What this system does (in simple words)

- You send a request to the server using an API.
- The server saves your request in a database and immediately replies, so you are not kept waiting.
- A background worker processes the request later.
- You can check the status anytime or cancel it if it is still running.

Why this design

- It prevents the same request from being processed twice.
- It handles failures safely and retries only when it makes sense.
- It works correctly even if many requests arrive at the same time.

Main building blocks

1) API Controller (ProcessingRequestsController)

- This is the front door.
- It checks your request, ensures you provided an Idempotency-Key, and sends back a response.
- It returns friendly errors if something is wrong (missing data, bad JSON, etc.).

2) Database Model (ProcessingRequest)

- This is the record stored in the database.
- It keeps the request data, status, attempts, timestamps, and any errors.
- Status values: pending, processing, retrying, completed, failed, cancelled.

3) Idempotency Service (RequestSubmission)

- This makes sure the same Idempotency-Key is not used twice with different data.
- If the same key is used with the same payload, it does not create duplicates.

4) Background Job (ProcessRequestJob)

- This is the worker that processes requests in the background.
- It locks the database row so two workers cannot process the same request.
- It retries only for retryable errors and stops for non-retryable ones.

5) Processor (RequestProcessing::Processor)

- This is the core business logic.
- It simulates real work and can intentionally fail in different ways for testing.

6) Error Types (RequestProcessingErrors)

- RetryableError: something temporary, should retry.
- NonRetryableError: permanent failure, do not retry.

7) JSON Hashing (CanonicalJson)

- Ensures the payload is hashed in a consistent way.
- This is how we detect "same key, different payload".

What cases we covered

- Duplicate requests with the same Idempotency-Key
- Same Idempotency-Key with different payload (conflict)
- Retryable failures with controlled retries
- Non-retryable failures without retry
- Cancellation while processing
- Concurrency and race conditions
- Slow processing and long-running jobs

HTTP responses you will see

- 202 Accepted: request saved and queued
- 200 OK: request completed or already known
- 400 Bad Request: missing or invalid input
- 409 Conflict: same idempotency key used with different payload
- 500 Internal Server Error: unexpected server issue

Principles we followed

- SOLID: clear responsibilities for controller, service, job, and processor
- ACID: database constraints and row locks keep data safe
- DRY: shared logic for hashing and serialization
- COC: standard Rails structure

How it helps in real life

- You never process the same request twice.
- Users can safely retry calls without creating duplicates.
- The system keeps working even if parts fail temporarily.